

DISEÑO Y DESARROLLO DE SISTEMAS DE INFORMACIÓN

SEMINARIO 2

IMPLEMENTACIÓN DE UNA APLICACIÓN DE GESTIÓN DE UN RESTAURANTE



**UNIVERSIDAD
DE GRANADA**

**Antonio Cantillo Molina
Leandro Jorge Fernández Vega
Elena Abreu Fernández
Elena Gallardo Caro
Pablo Bolaños Martínez**

ÍNDICE

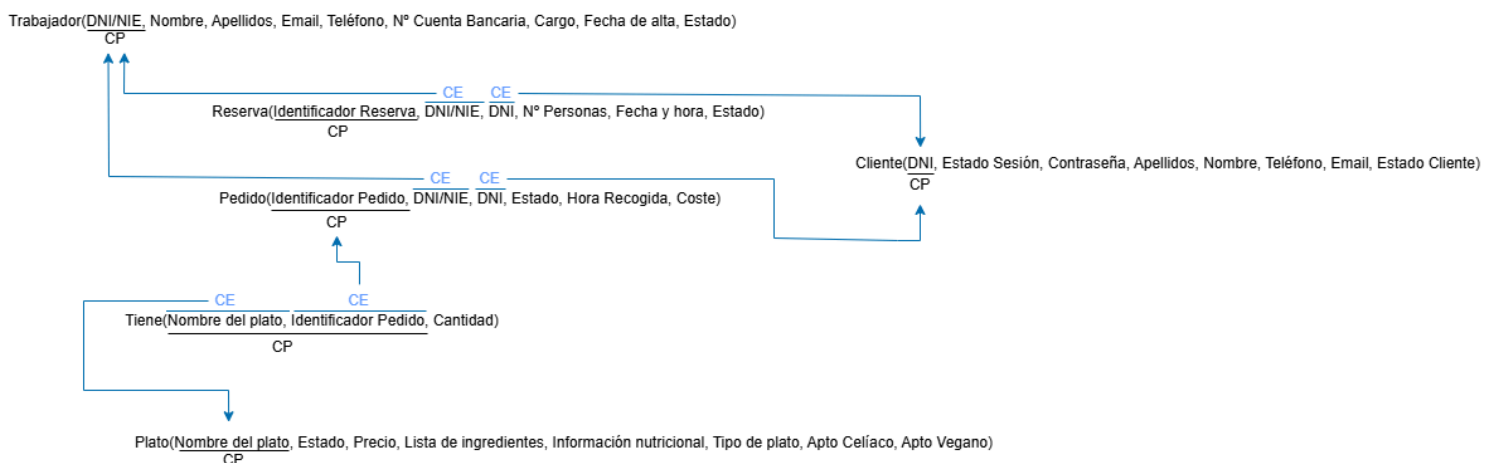
1. Información del Grupo	3
a. Gestión de personal (Antonio Cantillo Molina)	4
b. Gestión de clientes (Elena Abreu Fernández)	10
c. Gestión de platos (Pablo Bolaños Martínez)	14
d. Gestión de pedidos (Elena Gallardo Caro)	20
e. Gestión de reservas (Leandro Jorge Fernández Vega)	24

1. Información del Grupo

Grupo Pequeño (prácticas y seminarios)	A1
Grupo de trabajo	Uwuntu
Integrantes	Antonio Cantillo Molina Leandro Jorge Fernández Vega Elena Abreu Fernández Elena Gallardo Caro Pablo Bolaños Martínez

2. Introducción

Se propone la implementación de una aplicación de gestión de un restaurante, usando herramientas de alto nivel. Para ello, partimos del siguiente diseño:



3. Software Utilizado

Para la realización de esta implementación se ha utilizado *Oracle Apex*, una herramienta de alto nivel para crear aplicaciones, con bases de datos subyacentes. Es una herramienta en la web que no requiere la instalación de ningún programa.

Para aprender ciertas nociones básicas de esta herramienta, hemos utilizado el tutorial: <https://www.youtube.com/watch?v=o0ox1OiQjck&t=7s>, del canal oficial de Oracle.

4. Estructura de la Aplicación

a. Gestión de personal (Antonio Cantillo Molina)

Para la gestión de personal se ha decidido implementar el requisito funcional **RF1.1 “Registrar a un nuevo trabajador”** de tabla:

Requisito funcional			
Número de referencia	RF1.1		
Nombre del requisito	Registrar a un nuevo trabajador.		
Descripción del requisito	Registra un nuevo trabajador del restaurante en el sistema rellenando su información personal con los datos de entrada proporcionados por el usuario.		
Entrada	Agente externo	Administrador	
	Requisito de datos	Número de referencia	RDE1.1
		Composición	• Nombre: Cadena no vacía de hasta 20 caracteres. • Apellidos: Cadena no vacía de hasta 40 caracteres. • DNI/NIE: Cadena no vacía de 8 dígitos seguido de una letra. • Email: Cadena no vacía de hasta 20 caracteres. • Teléfono: Cadena no vacía de 9 dígitos. • Número de cuenta bancaria: Cadena no vacía de hasta 40 dígitos, donde los del principio pueden ser letras que actúan como identificadores del país.

			Cargo: A seleccionar uno de la lista: (cocinero, camarero, repartidor, etc.).
Datos internos	Requisito de datos	Número de referencia	RDW1.1
		Descripción	- Nombre: Cadena no vacía de hasta 20 caracteres. - Apellidos: Cadena no vacía de hasta 40 caracteres. - DNI/NIE: Cadena no vacía de 8 dígitos seguido de una letra. - Email: Cadena no vacía de hasta 20 caracteres. - Teléfono: Cadena no vacía de 9 dígitos. - Número de cuenta bancaria: Cadena no vacía de hasta 40 dígitos, donde los del principio pueden ser letras que actúan como identificadores del país. - Cargo: A seleccionar uno de la lista: (cocinero, camarero, repartidor, etc.). - Fecha de alta en el restaurante: Formato DD-MM-AAAA - Estado: Booleano activado
Salida	Agente externo	Administrador	
	Requisito de datos	Número de referencia	RDS1.1
		Composición	Mensaje de confirmación de la adición.
Restricciones semánticas		Número de referencia	RS1.1.1
		Descripción	El DNI debe de tener formato válido y ser único. (RDW1.1) La condición debe de cumplirse, de lo contrario se devuelve un mensaje de error y no se añade el nuevo empleado.
		Número de referencia	RS1.1.2

		Descripción	El número de cuenta bancaria debe de tener formato válido y estar asociado a una entidad bancaria. (RDW1.1) La condición debe de cumplirse, de lo contrario se devuelve un mensaje de error y no se añade el nuevo empleado.
		Número de referencia	RS1.1.3
		Descripción	La fecha de alta debe tener formato válido y ser la actual. (RDW1.1) La condición debe de cumplirse, de lo contrario se devuelve un mensaje de error y no se añade el nuevo empleado.
		Número de referencia	RS1.1.4
		Descripción	El nuevo trabajador debe de tener un cargo asociado. (RDW1.1) La condición debe de cumplirse, de lo contrario se devuelve un mensaje de error y no se añade el nuevo empleado.

La tabla asociada a la entidad de trabajador en el sistema fue creada en la aplicación con el siguiente código:

```
-- 1. SUBSISTEMA TRABAJADOR --> Revisado y corregido
CREATE TABLE TRABAJADOR (
  DNI_NIE VARCHAR2(20) NOT NULL,
  NOMBRE VARCHAR2(100),
  APELLIDOS VARCHAR2(100),
  EMAIL VARCHAR2(100),
  TELEFONO VARCHAR2(20),
  NUM_CUENTA_BANCARIA VARCHAR2(50),
  CARGO VARCHAR2(50),
  FECHA_ALTA DATE DEFAULT TRUNC(SYSDATE) NOT NULL,
  ESTADO VARCHAR2(20) DEFAULT ON NULL 'TRUE',
  CONSTRAINT PK_TRABAJADOR PRIMARY KEY (DNI_NIE)
);
```

Vemos que definimos “DNI/NIE” como clave primaria de la tabla Trabajador. Nos aseguramos de que la clave primaria nunca es nula usando “NOT NULL”. En adición, tal y como se muestra en la tabla del requisito, tanto “fecha de alta” como “estado” son atributos que son creados por el propio sistema. Es decir, no forman parte de la información del requisito de entrada **RDE1.1**. Para estos valores, se establece también que no sean nulos y un valor por defecto:

- Para “fecha de alta”, la fecha del sistema del día actual.
- Para “estado” (que simboliza si el trabajador sigue empleado en el restaurante o ha sido despedido), el valor “True”. Se pone como default este valor, pues cuando se da de alta a un trabajador, se entiende que está empleado (valor “true”). Se pondría a “false” con funcionalidades como la de “Dar de baja a un trabajador”.

Ahora, dado que queremos implementar el requisito antes especificado, realizamos las creaciones necesarias en el entorno Oracle Apex para la creación de páginas, pestañas y botones funcionales.

Sin embargo, se han de tener en cuenta las restricciones semánticas específicas del requisito implementado. Para ello, introducimos “triggers” o disparadores para controlar las acciones que se realizan y que los datos que, en este caso, se introducen, son correctos:

- El primer disparador comprueba que el campo de DNI/NIE no es nulo o la cadena vacía, haciendo saber al usuario, en su caso, que debe rellenar el campo. Esto es esencial pues el DNI/NIE actúa como clave primaria de la tabla Trabajador. Además, comprueba que el DNI/NIE introducido tiene formato válido. (Cumpliendo así **RS1.1.1**).

```
-- TRIGGER 1: Validar DNI en TRABAJADOR (RS1.1.1)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_DNI_TRABAJADOR
BEFORE INSERT OR UPDATE ON TRABAJADOR
FOR EACH ROW
BEGIN
    IF :NEW.DNI_NIE IS NULL OR TRIM(:NEW.DNI_NIE) = '' THEN
        RAISE_APPLICATION_ERROR(-20002, 'Error: Introduzca DNI/NIE del trabajador.');
```

- El segundo disparador se encarga de verificar, de nuevo, que tanto el campo de Número Cuenta Bancaria no es vacío, como que lo introducido se corresponde con una cuenta real (es decir, formato ESXXXX-XXXX-XXXX-XXXX-XXXX). (Cumpliendo así **RS1.1.2**).

```
-- TRIGGER 2: Validar NUM_CUENTA_BANCARIA en Trabajador (RS1.1.2)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_NUM_CUENTA_BANCARIA_TRABAJADOR
BEFORE INSERT OR UPDATE ON TRABAJADOR
FOR EACH ROW
BEGIN
    IF :NEW.NUM_CUENTA_BANCARIA IS NULL OR TRIM(:NEW.NUM_CUENTA_BANCARIA) = '' THEN
        RAISE_APPLICATION_ERROR(-20007,
            'Error: El número de cuenta bancaria no puede ser vacío.');
```

- El tercer disparador se encarga de verificar que el campo “Cargo” no se encuentra vacío, controlando así que todo trabajador tenga un cargo asignado. (Cumpliendo así **RS1.1.4**).

```
-- TRIGGER 3: Validar CARGO en Trabajador (RS1.1.4)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_CARGO
BEFORE INSERT OR UPDATE ON TRABAJADOR
FOR EACH ROW
BEGIN
    IF :NEW.CARGO IS NULL OR TRIM(:NEW.CARGO) = '' THEN
        RAISE_APPLICATION_ERROR(-20006,
            'Error: El trabajador ha de tener un cargo asignado obligatoriamente y no puede estar vacío.');
```


- El último disparador no hace referencia a ninguna restricción semántica. Simplemente se ocupa de evitar que el resto de campos de la entidad “Trabajador” no queden vacíos o con información incorrecta. Este disparador es **un disparador adicional a lo especificado en prácticas anteriores**, considerado para asegurar una mayor consistencia de los datos y añadir más funcionalidad a la aplicación.

```
-- ESTE TRIGGER ES ADICIONAL, LO PONGO PARA MEJORAR FUNCIONALIDAD (NO FORMA PARTE DE LAS RESTRICCIONES SEMANTICAS IMPUESTAS EN PR 1)

CREATE OR REPLACE TRIGGER TRG_VALIDAR_RESTO_CAMPOS_TRABAJADOR
BEFORE INSERT OR UPDATE ON TRABAJADOR
FOR EACH ROW
BEGIN
    IF :NEW.NOMBRE IS NULL OR TRIM(:NEW.NOMBRE) = '' THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: El nombre no puede ser vacío.');
```

Por último, destacar que no se ha implementado un disparador para el requisito **RS1.1.3**, puesto que el atributo “fecha de alta”, como hemos visto antes en la definición de la tabla, tiene como valor por defecto la fecha actual del sistema.

NOTA: En la parte de Page Design, se ha borrado el campo de estado. El usuario no debe rellenarlo, sino que es un atributo de control interno.

b. Gestión de clientes (Elena Abreu Fernández)

Para la gestión de clientes se ha decidido implementar el requisito funcional RF2.1, con la siguiente tabla:

Requisito funcional			
Número de referencia	RF2.1		
Nombre del requisito	Registrar cliente		
Descripción del requisito	Registra un nuevo cliente en el sistema rellenando su información con los datos de entrada proporcionados por el usuario		
Entrada	Agente externo	Cliente	
	Requisito de datos	Número de referencia	RDE2.1
		Composición	• DNI: cadena no vacía de 8 letras seguidas de un dígito • Nombre: cadena no vacía de hasta 20 caracteres • Apellidos: cadena no vacía de hasta 40 caracteres • Email: cadena no vacía de hasta 20 caracteres • Teléfono: cadena no vacía de 9 dígitos • Contraseña: cadena no vacía de entre 8-12 caracteres
Datos internos	Requisito de datos	Número de referencia	RDW2.1

		Descripción	• DNI: cadena no vacía de 8 letras seguidas de un dígito • Nombre: cadena no vacía de hasta 20 caracteres • Apellidos: cadena no vacía de hasta 40 caracteres • Email: cadena no vacía de hasta 20 caracteres • Teléfono: cadena no vacía de 9 dígitos • Contraseña: cadena no vacía de entre 8-12 caracteres • Fecha de alta : formato DD-MM-AAAA • Estado cliente: booleano, al crear el cliente se pone a true
Salida	Agente externo	Cliente	
	Requisito de datos	Número de referencia	RDS2.1
		Composición	• Mensaje de confirmación, secuencia de caracteres
Restricciones semánticas		Número de referencia	RS2.1.1
		Descripción	El DNI es único y tiene formato válido. (RDW2.1) Si alguna de estas condiciones no se cumple se devuelve un error.
		Número de referencia	RS2.1.2
		Descripción	La fecha de alta debe tener formato válido y ser la actual. (RDW2.1) Si alguna de estas condiciones no se cumple se devuelve un error.

Para ello hemos comenzado creando la siguiente tabla :

```
-- 2. SUBSISTEMA CLIENTE -> ya revisado y corregido
CREATE TABLE CLIENTE (
    DNI VARCHAR2(20) NOT NULL,
    ESTADO_SESION VARCHAR2(20) DEFAULT 'TRUE' NOT NULL,
    CONTRASENA VARCHAR2(100),
    APELLIDOS VARCHAR2(100),
    NOMBRE VARCHAR2(100),
    TELEFONO VARCHAR2(20),
    EMAIL VARCHAR2(100),
    ESTADO_CLIENTE VARCHAR2(20) DEFAULT 'TRUE' NOT NULL,
    FECHA_ALTA DATE,
    CONSTRAINT PK_CLIENTE PRIMARY KEY (DNI)
);
```

En la que hemos marcado DNI como la clave primaria, especificando que no puede ser nulo. Además, hemos marcado estado_sesión y estado_cliente como “true” por defecto, pues cuando se da de alta a un nuevo cliente la idea es que internamente se les asigne dichos valores, que sólo cambiarían en caso de cerrar sesión o darse de baja, respectivamente.

Se han implementado además los siguientes trigger para controlar las restricciones semánticas del requisito:

```
-- TRIGGER 5: Validar DNI en CLIENTE (RS2.1.1)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_DNI_CLIENTE
BEFORE INSERT OR UPDATE ON CLIENTE
FOR EACH ROW
BEGIN
    IF :NEW.DNI IS NULL OR TRIM(:NEW.DNI) = '' THEN
        RAISE_APPLICATION_ERROR(-20002, 'Error: Introduzca DNI/NIE del cliente.');
```

```
    ELSIF NOT REGEXP_LIKE(:NEW.DNI, '^[0-9]{8}[A-Z]$') THEN
        RAISE_APPLICATION_ERROR(-20002, 'Error: El DNI del cliente debe tener 8 dígitos y una letra mayúscula.');
```

```
    END IF;
END;
/
```

```

-- TRIGGER 4: Validar FECHA_ALTA en CLIENTE (RS2.1.2)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_FECHA_ALTA_CLIENTE
BEFORE INSERT OR UPDATE ON CLIENTE
FOR EACH ROW
BEGIN
    -- Si no se inserta FECHA_ALTA, poner la fecha actual
    IF :NEW.FECHA_ALTA IS NULL THEN
        :NEW.FECHA_ALTA := TRUNC(SYSDATE);
    -- Si se inserta, se comprueba que sea la fecha actual
    ELSIF TRUNC(:NEW.FECHA_ALTA) <> TRUNC(SYSDATE) THEN
        RAISE_APPLICATION_ERROR(-20005,'Error: La fecha de alta del cliente debe ser la fecha actual.');
```

Por último, en la parte de Page Design se marcan los campos estado_sesión y estado_cliente como hidden, pues no se busca que el cliente los rellene manualmente, sino que son atributos de gestión interna.

c. Gestión de platos (Pablo Bolaños Martínez)

Para la gestión de platos se ha decidido implementar el requisito funcional RF3.1:

Requisito funcional			
Número de referencia	RF3.1		
Nombre del requisito	Crear un nuevo plato		
Descripción del requisito	Registrar un nuevo plato en el sistema introduciendo su información básica. Una vez introducido, el sistema almacenará el plato y estará disponible para gestión y consulta.		
Entrada	Agente externo	Jefe	
	Requisito de datos	Número de referencia	RDE3.1
		Composición	• Nombre del plato (cadena alfanumérica no vacía de hasta 20 caracteres) • Lista de ingredientes (cadena de caracteres alfanuméricos) • Información nutricional (cadena de caracteres alfanuméricos) • Tipo de plato (tipo enumerado: 1 [entrante], 2 [plato principal], 3 [postre]) • Apto Celíaco (booleano) • Apto Vegano (booleano) • Precio (número decimal)
Datos internos	Requisito de datos	Número de referencia	RDW3.1
		Descripción	• Nombre del plato (cadena alfanumérica no vacía de hasta 20 caracteres)

			• Estado (booleano: true) • Lista de ingredientes (cadena de caracteres alfanuméricos) • Información nutricional (cadena de caracteres alfanuméricos) • Tipo de plato (tipo enumerado: 1 [entrante], 2 [plato principal], 3 [postre]) • Apto Celíaco (booleano) • Apto Vegano (booleano) • Precio (número decimal)
Salida	Agente externo	Jefe	
	Requisito de datos	Número de referencia	RDS3.1
		Composición	• Mensaje de confirmación del registro.
Restricciones semánticas		Número de referencia	RS3.1.1
		Descripción	• El nombre del plato debe tener un formato válido y ser único. (RDW3.1) • Si esta condición no se cumple, no se crea el plato y se devuelve un error.
		Número de referencia	RS3.1.2
		Descripción	• El precio debe ser un valor positivo (mayor que 0). (RDW3.1) • Si esta condición no se cumple, no se crea el plato y se devuelve un error.
		Número de referencia	RS3.1.3
		Descripción	• Si un plato se marca como apto para celíacos (true), ninguno de sus ingredientes puede contener gluten (trigo, cebada, espelta, etc.). (RDW3.1) • Si esta condición no se cumple, no se crea el plato y se devuelve un error.
		Número de	RS3.1.4

		referencia	
		Descripción	• Si un plato se marca como vegano (true), ninguno de sus ingredientes puede ser de origen animal (carne, pescado, huevo, lácteos, etc.). (RDW3.1) • Si esta condición no se cumple, no se crea el plato y se devuelve un error.

Para ello hemos comenzado creando la siguiente tabla :

```
-- 3. SUBSISTEMA PLATO
CREATE TABLE PLATO (
  NOMBRE_PLATO VARCHAR2(100) NOT NULL,
  ESTADO VARCHAR2(20) DEFAULT ON NULL 'TRUE',
  PRECIO NUMBER(10,2),
  LISTA_INGREDIENTES VARCHAR2(4000),
  INFORMACION_NUTRICIONAL VARCHAR2(4000),
  TIPO_DE_PLATO VARCHAR2(50),
  APTO_CELIACO VARCHAR2(1) CHECK (APTO_CELIACO IN ('S', 'N')),
  APTO_VEGANO VARCHAR2(1) CHECK (APTO_VEGANO IN ('S', 'N')),
  CONSTRAINT PK_PLATO PRIMARY KEY (NOMBRE_PLATO)
);
```

En la tabla PLATO hemos definido NOMBRE_PLATO como la clave primaria, ya que no pueden existir dos platos que se llamen igual. Además, hemos configurado el campo ESTADO para que tenga el valor 'TRUE' por defecto. De esta forma, al dar de alta un plato, este aparece automáticamente como "disponible" sin que tengamos que seleccionarlo manualmente. En el Page Designer hemos ocultado este campo para que no aparezca, ya que es un atributo de control interno que no debe manipular el usuario al rellenar el formulario.

Para el campo TIPO_DE_PLATO, aunque en la base de datos se guarda como un texto (VARCHAR2), hemos decidido controlar la entrada de datos desde la interfaz. En el Page Designer, hemos convertido este campo en una lista de selección (Select List) con las opciones fijas: 'Entrante', 'Plato principal' y 'Postre'. Con esto nos aseguramos de que el output sea siempre correcto y evitamos que el usuario pueda cometer errores ortográficos al escribir el tipo.

Hemos definido el precio como un número decimal (NUMBER(10,2)) para permitir costes exactos. Además, en el Page Designer hemos aplicado una máscara de formato específica. Esto soluciona un problema visual habitual: permite introducir y visualizar correctamente precios menores a la unidad (como 0.5) y añade automáticamente el símbolo del dólar (\$), facilitando la lectura sin alterar el dato numérico.

Para los campos APTO_CELIACO y APTO_VEGANO, hemos restringido los valores en la propia tabla mediante un CHECK IN ('S', 'N'). De esta forma, estandarizamos la respuesta a un formato simple de sí o no.

Para controlar las restricciones semánticas más complejas que no se pueden solucionar solo con la estructura de la tabla, hemos implementado los siguientes triggers:

```
-- TRIGGER 1: Validar Nombre del Plato (RS3.1.1 - Formato)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_NOMBRE_PLATO
BEFORE INSERT OR UPDATE ON PLATO
FOR EACH ROW
BEGIN
    -- Comprobamos que no sea nulo, vacío o solo espacios
    IF :NEW.NOMBRE_PLATO IS NULL OR TRIM(:NEW.NOMBRE_PLATO) = '' THEN
        RAISE_APPLICATION_ERROR(-20010, 'Error: El nombre del plato no puede estar vacío.');
```

- Este trigger comprueba que el NOMBRE_PLATO no esté vacío ni sea nulo, obligando a identificar el plato correctamente.

```
-- TRIGGER 2: Validar Precio Positivo (RS3.1.2)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_PRECIO_PLATO
BEFORE INSERT OR UPDATE ON PLATO
FOR EACH ROW
BEGIN
    -- Si se introduce un precio, debe ser mayor que 0
    IF :NEW.PRECIO IS NOT NULL AND :NEW.PRECIO <= 0 THEN
        RAISE_APPLICATION_ERROR(-20011, 'Error: El precio del plato debe ser mayor que 0.');
```

- También verificamos que el PRECIO sea un número estrictamente positivo (mayor que 0), para evitar errores de facturación.

```
-- TRIGGER 3: Validar Apto Celiacos (RS3.1.3)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_CELIACO
BEFORE INSERT OR UPDATE ON PLATO
FOR EACH ROW
BEGIN
    -- Si el plato es apto para celiacos ('S'), buscamos palabras prohibidas en los ingredientes
    -- La 'i' en regexp significa que no distingue mayúsculas de minúsculas
    IF :NEW.APTO_CELIACO = 'S' THEN
        IF REGEXP_LIKE(:NEW.LISTA_INGREDIENTES, 'trigo|cebada|centeno|avena|gluten|harina|pan|pasta', 'i') THEN
            RAISE_APPLICATION_ERROR(-20012, 'Error: Un plato apto para celiacos no puede contener gluten (trigo, pan, harina, etc).');
```

- Si se marca el plato como apto para celíacos ('S'), el trigger busca en la LISTA_INGREDIENTES palabras prohibidas como 'gluten', 'trigo', 'harina' o 'pan'. Si las encuentra, bloquea el registro.

```

-- TRIGGER 4: Validar Apto Vegano (RS3.1.4)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_VEGANO
BEFORE INSERT OR UPDATE ON PLATO
FOR EACH ROW
BEGIN
    -- Si el plato es vegano ('S'), buscamos palabras de origen animal
    IF :NEW.APTO_VEGANO = 'S' THEN
        IF REGEXP_LIKE(:NEW.LISTA_INGREDIENTES, 'carne|pescado|huevo|leche|queso|miel|nata|pollo|jamon|atun', 'i') THEN
            RAISE_APPLICATION_ERROR(-20013, 'Error: Un plato vegano no puede contener ingredientes de origen animal.');
        END IF;
    END IF;
END;
/

```

- De igual forma, si se indica que es vegano ('S'), el sistema revisa que no existan ingredientes de origen animal (como 'carne', 'pescado', 'huevo', 'leche', etc.) en la lista.

d. Gestión de pedidos (Elena Gallardo Caro)

Requisito funcional			
Número de referencia	RF4.1		
Nombre del requisito	Crear nuevo pedido		
Descripción del requisito	Añade un pedido dentro del sistema, a partir de los datos proporcionados por el usuario (DNI). Creando un identificador para el nuevo pedido y asociándolo a un trabajador		
Entrada	Agente externo	Cliente	
	Requisito de datos	Número de referencia	RDE4.1
		Composición	• DNI del cliente que lo realiza (cadena de 8 números seguida de un carácter). • Lista de nombres de plato
Datos internos	Requisito de datos	Número de referencia	RDW4.1
		Descripción	• Identificador del pedido (cadena numérica no vacía que se asignará internamente en el sistema a cada pedido) • DNI de trabajador (cadena numérica no vacía) • DNI del cliente (cadena numérica no vacía). • Nombre del plato (cadena numérica no vacía). • Estado del pedido (por defecto se inicializará a pedido) • Hora de recogida del pedido (hora formato HH:MM). • Coste del pedido.
Salida	Agente	Cliente y Trabajador	

	externo		
	Requisito de datos	Número de referencia	RDS4.1
		Composición	• Mensaje de confirmación alfanumérico que recibe el cliente. • Identificador del pedido • Lista con los nombres de plato. • Estado del pedido (por defecto se inicializará a pedido) • Hora de recogida del pedido (hora formato HH:MM). • Coste del pedido.
Restricciones semánticas		Número de referencia	RS4.1.1
		Descripción	• El DNI del cliente tiene que tener formato válido y ser único. (RDW4.1)
		Número de referencia	RS4.1.2
		Descripción	• El cliente debe de estar identificado para crear un pedido. (RDW4.1)
		Número de referencia	RS4.1.3
		Descripción	• Los nombres de los platos deben existir. (RDW4.1)
		Número de referencia	RS4.1.4
		Descripción	• La fecha y hora debe ser posterior a la una hora después de la actual (RDW4.1).

Para ello hemos comenzado creando la siguiente tabla :

```
-- 4. SUBSISTEMA PEDIDO
CREATE TABLE PEDIDO (
  ID_PEDIDO NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY,
  DNI_NIE_TRABAJADOR VARCHAR2(20),
  DNI_CLIENTE VARCHAR2(20) NOT NULL,
  ESTADO VARCHAR2(20) DEFAULT ON NULL 'Pendiente' CHECK (ESTADO IN ('Pendiente', 'Confirmada', 'Cancelada')),
  HORA_RECOGIDA TIMESTAMP,
  COSTE NUMBER(10,2),
  CONSTRAINT PK_PEDIDO PRIMARY KEY (ID_PEDIDO),
  CONSTRAINT FK_PEDIDO_TRABAJADOR FOREIGN KEY (DNI_NIE_TRABAJADOR) REFERENCES TRABAJADOR(DNI_NIE),
  CONSTRAINT FK_PEDIDO_CLIENTE FOREIGN KEY (DNI_CLIENTE) REFERENCES CLIENTE(DNI)
);

CREATE TABLE PLATOS_TIENE_PEDIDO (
  ID_PEDIDO NUMBER NOT NULL,
  NOMBRE_PLATO VARCHAR2(100) NOT NULL,
  CANTIDAD NUMBER(5) NOT NULL,
  CONSTRAINT PK_TIENE PRIMARY KEY (ID_PEDIDO, NOMBRE_PLATO),
  CONSTRAINT FK_TIENE_PEDIDO FOREIGN KEY (ID_PEDIDO) REFERENCES PEDIDO(ID_PEDIDO),
  CONSTRAINT FK_PLATOS_TIENE FOREIGN KEY (NOMBRE_PLATO) REFERENCES PLATO(NOMBRE_PLATO)
);
```

En la que hemos marcado ID_PEDIDO como la clave primaria, especificando que no puede ser nulo. Además, tenemos dos claves externas que son los dos dnis de trabajador y cliente. Y, el estado del pedido se creará como 'pendiente' por defecto hasta que esté Confirmado o se Cancele el pedido. Para controlar las restricciones semánticas de pedido hemos implementado los siguientes trigger:

```
-- TRIGGER 3: Validar HORA RECOGIDA en PEDIDO (RS4.1.4)
CREATE OR REPLACE TRIGGER TRG_VALIDAR_HORA_RECOGIDA
BEFORE INSERT OR UPDATE ON PEDIDO
FOR EACH ROW
BEGIN
    IF :NEW.HORA_RECOGIDA <= SYSDATE +(1/24) THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: La fecha del pedido debe ser posterior de una más a la actual.');
```

Este comprueba el requisito RS4.1.4 La fecha y hora debe ser posterior a la una hora después de la actual.

Para el resto sería redundante hacer un trigger porque las hemos solucionado en su creación:

- **RS4.1.1 El DNI del cliente tiene que tener formato válido y ser único.**
- **RS4.1.2 El cliente debe de estar identificado para crear un pedido.**

En la tabla PEDIDO hemos creado DNI_CLIENTE como NOT NULL que obliga a rellenar este atributo para crear el pedido y además es clave externa de cliente, donde ya hemos controlado el formato.

- **RS4.1.3 Los nombres de los platos deben existir**

CONSTRAINT FK_PLATOS_TIENE_PEDIDO FOREIGN KEY (NOMBRE_PLATO) REFERENCES PLATO(NOMBRE_PLATO) teniendo en cuenta esta línea, siendo clave externa ya estaría solucionado este requisito.

Por último, en la parte de Page Design se marcan el campo ESTADO como oculto, pues no se pretende que el usuario lo rellene, sino que es un atributo de control interno.

e. Gestión de reservas (Leandro Jorge Fernández Vega)

Para la gestión de reservas, se ha decidido implementar el requisito:

Requisito funcional			
Número de referencia	RF5.1		
Nombre del requisito	Crear reserva.		
Descripción del requisito	Crear una nueva reserva en el sistema. El estado será pendiente hasta que se realice el pago de la fianza.		
Entrada	Agente externo	Cliente	
	Requisito de datos	Número de referencia	RDE5.1
		Composición	• DNI del cliente : cadena no vacía de 8 dígitos seguida de una letra. • Número de personas: entero • Fecha y hora: cadena de hasta 30 caracteres.
Datos internos	Requisito de datos	Número de referencia	RDW5.1
		Descripción	• Identificador de la reserva: cadena no vacía de 10 dígitos. • DNI del cliente : cadena no vacía de 8 dígitos seguida de una letra. • DNI del trabajador: cadena no vacía de 8 dígitos seguida de una letra • Número de personas: entero

			- Fecha y hora: cadena de hasta 30 caracteres. - Estado: cadena de hasta 15 caracteres.
Salida	Agente externo	Cliente	
	Requisito de datos	Número de referencia	RDS5.1
		Composición	- Identificador de la reserva: cadena no vacía de 10 dígitos. - Mensaje de confirmación de la reserva.
Restricciones semánticas		Número de referencia	RS5.1.1
		Descripción	- Solo puede haber una reserva en el sistema con el mismo identificador (RDW5.1). - Si esta condición no se cumple, se devuelve un error y no se inserta reserva en el sistema.
		Número de referencia	RS5.1.2
		Descripción	- Los DNIs deben tener formato válido (RDW5.1). - Si esta condición no se cumple, se devuelve un error y no se inserta reserva en el sistema.
		Número de referencia	RS5.1.3
		Descripción	- La fecha y hora debe ser posterior a la actual (RDW5.1). - Si esta condición no se cumple, se devuelve un error y no se inserta reserva en el sistema.

Para ello, hacemos uso de la siguiente tabla:

```
-- 5. SUBSISTEMA RESERVA
CREATE TABLE RESERVA (
  ID_RESERVA NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY,
  DNI_NIE_TRABAJADOR VARCHAR2(20),
  DNI_CLIENTE VARCHAR2(20),
  NUM_PERSONAS NUMBER DEFAULT ON NULL 1,
  FECHA_HORA TIMESTAMP DEFAULT ON NULL SYSDATE,
  ESTADO VARCHAR2(20) DEFAULT ON NULL 'Pendiente' CHECK (ESTADO IN ('Pendiente', 'Confirmada', 'Cancelada')),
  CONSTRAINT PK_RESERVA PRIMARY KEY (ID_RESERVA),
  CONSTRAINT FK_RESERVA_TRABAJADOR FOREIGN KEY (DNI_NIE_TRABAJADOR) REFERENCES TRABAJADOR(DNI_NIE),
  CONSTRAINT FK_RESERVA_CLIENTE FOREIGN KEY (DNI_CLIENTE) REFERENCES CLIENTE(DNI)
);
```

Para implementar las restricciones semánticas, se hace uso de un disparador:

```
-- 2. DISPARADOR PARA VALIDAR LA RESERVA
CREATE OR REPLACE TRIGGER TRG_VALIDACIONES_RESERVA
BEFORE INSERT OR UPDATE ON RESERVA
FOR EACH ROW
BEGIN
  -- VALIDACIÓN 1: Que los campos no sean NULOS
  IF :NEW.DNI_NIE_TRABAJADOR IS NULL OR :NEW.DNI_CLIENTE IS NULL THEN
    RAISE_APPLICATION_ERROR(-20004, 'Error: Es obligatorio indicar el DNI del Cliente y del Trabajador.');
```

```
  END IF;

  -- VALIDACIÓN 2: Formato del DNI del TRABAJADOR (8 números + 1 Letra Mayúscula)
  IF NOT REGEXP_LIKE(:NEW.DNI_NIE_TRABAJADOR, '^[0-9]{8}[A-Z]$') THEN
    RAISE_APPLICATION_ERROR(-20002, 'Error RS5.1.2: El DNI del trabajador no tiene un formato válido (12345678A).');
```

```
  END IF;

  -- VALIDACIÓN 3: Formato del DNI del CLIENTE (8 números + 1 Letra Mayúscula)
  IF NOT REGEXP_LIKE(:NEW.DNI_CLIENTE, '^[0-9]{8}[A-Z]$') THEN
    RAISE_APPLICATION_ERROR(-20003, 'Error RS5.1.2: El DNI del cliente no tiene un formato válido (12345678A).');
```

```
  END IF;

  -- VALIDACIÓN 4: La Fecha debe ser FUTURA (RS5.1.3)
  IF :NEW.FECHA_HORA < SYSDATE THEN
    RAISE_APPLICATION_ERROR(-20001, 'Error RS5.1.3: La fecha de la reserva debe ser posterior al momento actual.');
```

```
  END IF;
END;
```

```
/
```

Explicación

- Para que se cumplan las propiedades de clave primaria, la no nulidad y unicidad de los identificadores de reserva en el sistema se asegura dejando al sistema que los cree automáticamente cada vez que se inserte una tupla. Por tanto, al rellenar los campos de una tupla desde la interfaz no se muestra el campo para indicar un identificador de reserva.
- La reserva se inicializa a estado “Pendiente” a no ser que se indique otro estado.
- La fecha se inicializa a la de hoy si no se indica ninguna otra.
- El número de personas se inicializa a 1 en caso de que no se indique un número.
- Las claves externas referencian DNIs de clientes y trabajadores dados de alta en el sistema.
- El disparador implementa las restricciones semánticas mencionadas, aparte de controlar que las claves externas no sean nulas.

Cambios en la implementación

La facilidad de uso de *Oracle Apex* permite implementar este requisito de forma directa, junto con el requisito de modificación y eliminación. Sin embargo, nos centraremos en el descrito.

El principal cambio radica en el uso que se haría de la aplicación. Según el pensamiento inicial, era el usuario el que crearía una reserva. Sin embargo, la implementación solo permite que lo haga un administrador, que seleccionará manualmente el DNI del cliente y del trabajador, para poder asociarlos a una reserva. La salida también supone un cambio frente al requisito, ya que se muestra toda la tupla insertada. El agente externo de salida será el administrador.

Sin embargo, añadimos una restricción que implique que las claves externas no puedan ser nulas, para que no se inserte una reserva en el sistema sin un cliente o trabajador asociado.

Por otra parte, inicializamos el número de personas a 1 si no se indica otro número, y la fecha a la de hoy si no se especifica ninguna.

Finalmente, aunque no figure en las restricciones semánticas, por propia seguridad del sistema, se comprueba que los estados solo sean “Pendiente”, “Confirmada” o “Cancelada”. No es realmente necesario, ya que al insertar una reserva desde la interfaz, solo se pone a disposición una lista con los estados disponibles.